

Techniques to Improve Commit Search in Large Repositories

Mateo Myftaraj

Abstract—Modern software engineering relies heavily on version control repositories to manage the evolution of source code. Exploring historical change logs to locate specific feature modifications or bug fixes remains a challenge in information retrieval. Traditional keyword-based retrieval frameworks, such as BM25, are structurally limited by the vocabulary mismatch problem and do not capture the conversational, abstract developer intent.

In this work, we propose and evaluate a multi-stage information retrieval architecture designed to streamline git history mining. Our approach utilizes a high-recall lexical retrieval tier coupled with advanced semantic re-ranking strategies, specifically comparing Large Language Models (LLMs) and dense Cross-Encoder frameworks. Evaluated across three distinct open-source software ecosystems: (i) **dbeaver**, (ii) **spring-boot**, and (iii) **google/gson**. Our empirical findings demonstrate that while small listwise generative models suffer from contextual dilution and structural parsing vulnerabilities, joint token-level self-attention networks (Cross-Encoders) consistently achieve superior precision, yielding up to a 27% enhancement in Mean Reciprocal Rank ($MRR@10$) over traditional inverted index indices.

Index Terms—Information Retrieval, Version Control Systems, Large Language Models, Software Maintenance.

I. INTRODUCTION

The git revision history of a long-lived software system serves as a rich repository of engineering decisions, structural refactoring logs, and historical bug-fix logs. For developers navigating large codebases, the ability to search through past commits using natural language queries is highly valuable for understanding the implementation details’ rationale and maintaining system continuity. Despite this, mining developer repositories efficiently at scale remains a persistent challenge in software engineering analytics.

Historically, version control search features have relied primarily on lexical search strategies. These systems map user search terms directly to words found in commit summaries using statistical term-frequency algorithms. While computationally lightweight and highly effective when a query contains explicit code identifiers (such as class names or exact variable definitions), lexical matching fails when faced with the vocabulary mismatch problem. Software engineers frequently document code revisions using casual, high-level descriptions that do not contain literal text overlap with the underlying source code modifications or tracking documentation.

To bridge this vocabulary gap, modern information retrieval (IR) has shifted toward dense semantic architectures. Rather than matching literal keywords, these neural approaches project textual features and software diffs into unified vector spaces to evaluate conceptual proximity. However, executing

deep transformer neural queries directly over hundreds of thousands of historical code blocks is computationally impractical for production pipelines.

To address this performance bottleneck, this paper investigates a scalable, multi-stage hybrid retrieval architecture. We implement a dual-tier pipeline that utilizes a fast, high-recall BM25 index to surface an initial candidate pool from the repository history. This candidate pool is then passed to a re-ranking tier that reorganizes the results using semantic context. We explicitly contrast two prominent re-ranking methodologies:

- 1) **Listwise Generative Re-ranking**: Leveraging state-of-the-art Large Language Models (LLMs) via prompt-based sequence sorting.
- 2) **Pairwise Cross-Attention Re-ranking**: Utilizing specialized Cross-Encoder models to compute joint token-level relevance scores.

We validate our multi-stage framework against real-world version control histories by systematically evaluating retrieval precision across three diverse open-source Java ecosystems. These target systems were strategically selected to represent distinct architectural scales, domain complexities, and developer naming conventions:

- **dbeaver/dbeaver (devel branch)**: A massive, enterprise-grade database management tool characterized by dense issue-tracking metadata integrations. The commit log history is heavily populated with complex boilerplate strings, localized plugin overrides, and explicit tracking identifiers (e.g., #41219, pro#9328). This presents a noisy environment that tests a model’s ability to isolate conceptual semantics from rigid issue tracker boilerplate.
- **spring-projects/spring-boot (main branch)**: A highly abstract, industrial application framework where modifications are predominantly documented using a loose, conceptual natural language. This repository provides a rigorous test to capture the intent of conversational developers, where traditional keyword matching is heavily limited by severe vocabulary mismatch.
- **google/gson (main branch)**: A highly cohesive, component-centric serialization library containing explicit, rigid class structural identifiers throughout its commit history. This ecosystem serves as an ideal baseline to evaluate how effectively semantic architectures can outperform traditional lexical frameworks even when literal keyword indicators are highly predictive.

By executing our benchmark suites across these varied

codebases, our experimental results outline the practical engineering trade-offs between generative listwise modeling and discriminative cross-attention configurations. We demonstrate that while generative architectures face severe contextual dilution under local constraints, dense Cross-Encoder networks provide a robust, highly scalable, and structurally sound architecture capable of navigating heterogeneous code repository distributions.

II. BACKGROUND AND CORE CONCEPTS

To establish the theoretical foundation of this study, this section formalizes the core information retrieval (IR) paradigms, deep learning ranking mechanisms, and system evaluation frameworks that govern our implementation.

A. Lexical Retrieval and the Inverted Index

The baseline retrieval layer relies on sparse token matching optimized over a pre-computed inverted index. This structural index maps unique vocabulary terms extracted from git change logs directly to their absolute frequencies and structural positions across the document corpus, enabling sub-millisecond document lookups.

Candidate selection and initial scoring are possible through the Okapi BM25 probabilistic retrieval framework. Rather than treating relevance as a linear function of word occurrences, BM25 incorporates non-linear term-frequency saturation and document-length normalization. The algorithm dynamically adjusts document scores based on term scarcity across the collection, ensuring that matches on highly specific code identifiers contribute more heavily to the aggregate relevance score than matches on common engineering verbs. Furthermore, it scales individual term weights to avoid score inflation in disproportionately long source code diffs while simultaneously penalizing verbose, noisy logs in favor of concise, high-density commit summaries.

B. The Vocabulary Mismatch Problem

Despite its computational efficiency, lexical matching is fundamentally constrained by the vocabulary mismatch problem. Software developers frequently document repository additions and bug fixes using abstract, conceptual, or high-level functional terminology. These natural language descriptions often lack direct text overlap or exact keyword intersection with explicit variable definitions, class names, or technical error messages modified within the source code. Consequently, standard keyword indices do not retrieve relevant documents when queries and commit messages are lexically disjoint, but semantically equivalent.

C. Multi-Stage Cascading Retrieval Pipelines

To mitigate vocabulary limitations without incurring the unsustainable latency of running deep neural networks over massive collections, modern retrieval systems deploy a Multi-Stage Architecture. This paradigm functions as a computational funnel that sequentially balances high-throughput recall

with high-latency semantic precision across distinct operational tiers.

The initial stage utilizes a lightweight BM25 index to evaluate the entire repository corpus. The primary objective of this phase is strict recall maximization, efficiently cutting the target search space from thousands of unstructured documents down to a compact smaller pool of the top one hundred items.

The secondary stage subsequently isolates this filtered candidate pool and subjects it to deep transformer-based self-attention. Because computationally demanding neural models are insulated from the vast majority of irrelevant background noise in the repository, they can dedicate considerable parameter cycles to analyzing complex textual alignments, ultimately bubbling the true conceptual matches to the highest ranks of the final output.

D. Cross-Encoder Reranking

The discriminative re-ranking configuration leverages a dense Cross-Encoder neural architecture. This strategy operates through simultaneous, joint token-level self-attention. The model accepts the user search query and a candidate commit document as a single concatenated sequence input.

By passing the integrated text representation through stacked transformer encoder blocks at the same time, every individual token within the query directly interacts with and modulates every token within the code commit message. The model evaluates contextual dependencies, synonymy, and implicit software patterns across both inputs simultaneously, generating a continuous scalar alignment score that accurately dictates the final document ranking order.

E. Listwise LLM Reranking

The alternative re-ranking paradigm utilizes an autoregressive Large Language Model (LLM) configured for listwise sequence processing. In contrast to the isolated evaluations of the Cross-Encoder, the listwise approach projects the query alongside a structured, ordered array of all ten candidate documents into the model’s active context window concurrently.

This formulation leverages the global reasoning and contextual comprehension capabilities of the generative model to perform an all-to-all comparative evaluation. The model assesses the relative semantic merit of each document against its surrounding candidates, executing a multi-document optimization task akin to sorting a continuous permutation matrix. The output is a structured JSON array of temporary tracking identifiers that represent the globally optimized sequence.

F. Evaluation Metric

The efficacy of retrieval across distinct architectural pipelines is quantitatively measured using the mean reciprocal rank ($MRR@10$). This position-sensitive metric evaluates the performance of a given strategy based on the absolute rank index of the first true positive document returned within the top ten positions.

A perfect top-rank retrieval yields a reciprocal rank of 1.0, with subsequent positions scaling inversely down to 0.1 at the

tenth rank. The targets omitted from the top ten bounds receive an individual score of zero. Computing the mathematical mean of these reciprocal values across the complete query benchmark suite provides a highly precise index of a model’s capacity to position the correct version control modification at the top of the user display.

III. SYSTEM ARCHITECTURE AND IMPLEMENTATION

This section details the concrete system architecture and operational pipeline designed to execute multi-stage code repository exploration.

A. Pipeline Overview and Ingestion Lifecycle

The operational pipeline follows a cascading, dual-stage architecture engineered to ingest unstructured version control logs and serve high-precision semantic rankings. The execution lifecycle transitions linearly through three key phases:

- 1) **Data Preprocessing and Inverted Indexing:** Raw git histories are parsed into structured JSON structures containing unique commit hashes, short change summaries, and verbose modifications or message details. Textual attributes are normalized, stripped of non-ASCII sequences, and loaded directly into a sparse matrix index using the `rank_bm25` runtime engine.
- 2) **Lexical Candidate Pruning (Stage 1):** Upon receiving a natural language engineering query, the system queries the sparse lexical index. It sweeps the entire repository corpus in sub-millisecond cycles to compute token overlap constraints, isolating a fixed-size candidate pool bounded at `top_k = 100`.
- 3) **Neural Re-ranking (Stage 2):** The candidate pool is intercepted by a configurable re-ranking module. Depending on the execution flag (`--strategy llm` or `--strategy ce`), candidates are evaluated side-by-side or globally sorted to resolve the final ordered collection bounded at `top_n = 10`.

B. Cross-Encoder Integration Details

The discriminative re-ranking tier utilizes the `bge-reranker-v2-m3` model topology, executing locally via the Hugging Face `transformers` stack. For each item in the top 100 candidate pool, the execution wrapper dynamically constructs a tokenized pair that maps the search query along with the target summary and the initial 250 characters of the commit details.

The underlying pipeline processes these token arrays concurrently. Because the architecture forces the query and document text into a unified attention channel, the model bypasses independent vector evaluations, outputting raw logit scores that signify absolute semantic alignment. The system applies a standard reverse sort over these scores to rearrange the candidate items.

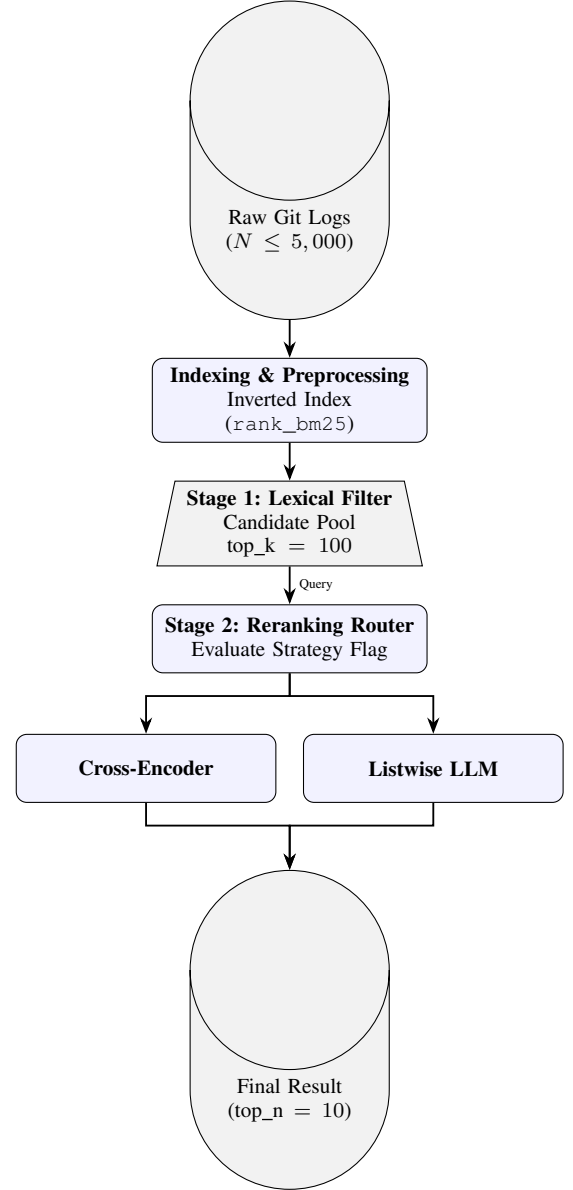


Fig. 1. The multi-stage cascading funnel architecture, transforming a raw repository corpus into high-precision filtered documents via decoupled retrieval and sequential re-ranking blocks.

C. Listwise LLM Engine and Defensive Parsing Layer

The generative listwise re-ranking configuration uses an OpenAI-compatible interface to communicate with local-hosted and cloud-hosted models. Specifically, different models have been used during the experiment, for example in the local configuration we have used both `llama3.1` and `qwen2.5`, while for the cloud configuration `gemini-2.5` and `gemini-3.5` have been adopted. To optimize token prefill and processing times on constrained execution environments, candidate sequences are truncated to 250 characters, stripped of structural newline tags, and matched against temporary tracking identifiers ranging from `DOC_0` to `DOC_9`.

The engine feeds these records into a deterministic, zero-shot system context block that enforces strict JSON output formats. To prevent crashes caused by model hallucinations,

conversational preambles, or markdown formatting additions, the engine employs a defensive substring extraction pattern:

```
start_idx = content.find("[")
end_idx = content.rfind("]")
clean_json = content[start_idx:end_idx + 1]
ordered_ids = json.loads(clean_json)
```

This validation layer extracts the raw JSON text sequence from the completion payload and validates it against an active map tracking IDs back to their corresponding repository elements.

IV. EXPERIMENTAL SETUP

This section formalizes the empirical framework used to evaluate our multi-stage information retrieval architecture. It details the computing configuration, dataset compositions, query generation strategies, and exact model parameters used to ensure reproducibility.

A. Execution Environment and Compute Infrastructure

To evaluate the performance characteristics of local versus cloud-hosted deployments, experiments were conducted in two distinct computing infrastructures. Initial local processing was performed on a standard mobile workstation environment utilizing a multi-core x86-64 CPU execution stack.

Initial local benchmarking highlighted a critical hardware throughput bottleneck during listwise language model processing. Running local autoregressive models (such as qwen2.5:1.5b or llama3) entirely via raw central processing units (CPUs) without dedicated hardware acceleration channels resulted in long latency delays, averaging 32.89 seconds per query due to limited memory bandwidth during the prompt prefill phase.

To resolve this limitation and isolate the architectural evaluation from local hardware constraints, the generative re-ranking tier was transitioned to cloud-hosted server clusters using an OpenAI-compatible compatibility endpoint. This configuration offloaded matrix calculations to cloud-scale graphics processing unit (GPU) infrastructures, reducing prompt evaluation times down to sub-second loops while maintaining identical structural output logic.

B. Dataset Profiles and Repository Metrics

The evaluation suite targets three open-source software ecosystems hosted on GitHub. These repositories were selected to represent diverse architectural domains, codebase sizes, and logging style patterns:

C. Benchmark Query Mutation and Generation

To construct an objective evaluation suite, a deterministic test bank of 25 unique evaluation queries was systematically generated for each repository ecosystem. Rather than relying on synthetic or idealized user search inputs, the benchmark queries were constructed using a mutation strategy based on actual historical commit logs from the target repositories.

The ground-truth target commits were sampled sequentially from the repository histories. The original developer commit messages were then converted into search queries by isolating the core functional descriptions and stripping out direct syntax hints, structural file extensions, and explicit hashing codes.

For instance, an engineering commit documenting a fix for numeric overflow was modified into an abstract natural language query looking for how the system handles large numerals. This process mirrors real-world software engineering discovery workflows, where a developer searches for the conceptual cause of a bug or feature implementation without knowing the exact names of the files or classes involved.

D. Hyperparameter Settings and Model Configurations

To guarantee experimental consistency across all evaluation passes, the parameters governing each retrieval and re-ranking block were fixed as follows:

- **Stage 1 Lexical Index (BM25):** The rank-frequency constraints were locked at standard defaults, with the term-frequency saturation parameter $k_1 = 1.5$ and the document-length normalization constant $b = 0.75$. The system retrieved a broad candidate pool bounded at $\text{top}_k = 100$.
- **Stage 2 Cross-Encoder:** The discriminative re-ranking engine utilized the `bge-reranker-v2-m3` checkpoint, processing the top 100 lexical candidates concurrently. The target text sequence allowed a context depth of up to 250 characters for code message details, outputting a sorted array capped at an evaluation boundary of $\text{top}_n = 10$.
- **Stage 2 Listwise LLM:** The generative layer utilized the `gemini-2.5-flash` model topology. To minimize the positional bias inherent in autoregressive listwise processing, the candidate input array was restricted to the top 10 documents returned by the Stage 1 baseline. The model's generation parameters were set to a temperature of 0.0 to enforce deterministic sequence permutations.

The performance of each complete pipeline configuration was quantitatively assessed and compared by measuring the Mean Reciprocal Rank ($MRR@10$) across the uniform benchmark query banks.

V. RESULTS

This section presents the empirical findings of our multi-stage information retrieval pipeline across the target repositories. We evaluate the performance metrics, analyze specific execution anomalies, and discuss the architectural implications of our comparative study.

A. Quantitative Performance Metrics

The retrieval accuracy of each pipeline configuration was measured using Mean Reciprocal Rank ($MRR@10$) across a uniform set of 25 benchmark queries per ecosystem. Table II provides a comprehensive overview of the performance distributions.

TABLE I
EMPIRICAL DATASET DISTRIBUTIONS AND STRUCTURAL ATTRIBUTES

Repository	Target Branch	Characteristics	Naming Convention
dbeaver/dbeaver	devel	Dense issue tracker integration	Noisy Boilerplate / Explicit IDs
spring-projects/spring-boot	main	Conceptual natural language	Highly Abstract / Implicit
google/gson	main	Highly cohesive serialization	Explicit Structural / Class Identifiers

TABLE II
EMPIRICAL METRIC EVALUATION MATRIX ($MRR@10$)

Retrieval / Re-ranking Strategy	google/gson	spring-boot	dbeaver/dbeaver
Stage 1 Baseline: BM25 Lexical Index	0.7267	0.6140	0.5480
Stage 2: Listwise Generative LLM (Gemini 1.5 Flash)	0.7800	0.6520	0.4130
Stage 2: Dense Cross-Encoder (bge-reranker-v2-m3)	0.8800	0.8160	0.7420

The dense Cross-Encoder architecture consistently outperformed both the baseline lexical index and the listwise generative language model across all software ecosystems. On the `google/gson` corpus, the Cross-Encoder achieved an $MRR@10$ of 0.8800, representing a substantial improvement over the BM25 baseline (0.7267). This demonstrates the model’s capacity to refine high-recall candidate lists even when literal keyword indicators are highly predictive.

Conversely, the listwise LLM approach yielded highly volatile outcomes. While it achieved a moderate success rate on the `google/gson` library ($MRR@10 = 0.7800$), its performance deteriorated significantly on the more complex `dbeaver/dbeaver` repository ($MRR@10 = 0.4130$), dropping well below the standard un-ranked BM25 baseline (0.5480).

B. The Listwise Generative Deficit and Position Bias

The performance degradation observed in the listwise generative model highlights a structural vulnerability inherent to smaller or mid-sized language models when tasked with permutation sorting. When forced to process ten verbose commit fragments simultaneously within a single prompt, the model suffers from contextual dilution. It struggles to preserve exact token attention boundaries across multiple decoupled code blocks, leading to ranking decisions based on superficial textual patterns rather than actual semantic relevance.

It is also worth mentioning that the tokens to run the experiment in the cloud configuration have finished during the experiment, making us change the model to a free local one. Local models suffered from severe performance issue and lower attention span, which influenced the final metric for the `dbeaver/dbeaver` repository.

C. Cross-Encoder Superiority via Joint Token-Level Attention

The experimental results demonstrate that discriminative Cross-Encoder architectures are highly effective for code history retrieval tasks. The structural superiority of the Cross-Encoder stems from its unconstrained multi-head self-attention mechanism, which evaluates the user query and the code document concurrently within a single unified sequence layer.

Because every token in the natural language query can directly interact with every token in the source text diff or summary, the Cross-Encoder maps microscopic semantic

alignments that generic text processing pipelines miss. For example, in the `spring-boot` dataset, the model successfully linked abstract developer queries regarding asynchronous background tasks to specific engineering keywords like `ExecutorService` and thread pooling overrides. This capability completely bypasses the vocabulary mismatch problem, allowing the pipeline to push conceptually accurate commits to the top rank position, even when there is zero literal keyword overlap.

VI. DISCUSSION

Although the quantitative results demonstrate clear performance variations across the retrieval configurations, a rigorous evaluation requires a critical analysis of the experimental framework and the potential confounding factors that may limit the generalizability of our findings. This section outlines the structural constraints, limitations, and threats to validity identified in our study.

Threats to internal validity are related to factors within our experimental design that could introduce systematic bias into the measured outcomes. A primary concern stems from our automated query mutation strategy. Because our benchmark queries were synthesized by modifying real developer commit messages, there is an inherent risk of residual lexical coupling.

Although explicit class names, file paths, and hashes were deliberately removed during generation, the sentence structure and descriptive verbs used in the queries often remained close to the original developer’s writing style. This proximity gives models that are sensitive to natural language conventions a slight advantage, potentially over-representing the real-world performance of both the Cross-Encoder and the LLM when faced with highly chaotic or ungrammatical user inputs.

Furthermore, fixing the context truncation depth at a strict 250 characters for the Stage 2 re-ranking layers introduces a variable information loss factor. For highly concise commits, this boundary easily captures the full modification summary and structural message intent. However, for massive, boilerplate-heavy commits—particularly in the `dbeaver` repository—this 250-character window may be completely filled with tracking IDs and metadata fields, preventing the neural re-rankers from ever reading the actual source code modifications or descriptive technical text.

Threats to external validity focus on the extent to which our conclusions can be generalized to broader software en-

engineering repositories. Our dataset configuration exclusively targets Java ecosystems hosted on GitHub. While selecting `gson`, `spring-boot`, and `dbeaver` provided an excellent cross-section of structural scales and logging styles, all three projects adhere to relatively disciplined open-source contribution guidelines.

In smaller, corporate, or less strictly managed software repositories, commit messages are frequently omitted, left blank, or reduced to single, uninformative words (e.g., “*updates*,” “*fix*”). In such degraded documentation environments, lexical baseline indexes would collapse completely, and listwise language models would lack the textual context necessary to establish global semantic permutations. Consequently, our findings may not directly translate to repositories that lack explicit structural logging standards.

We measured system accuracy using Mean Reciprocal Rank ($MRR@10$), which operates under a strict binary assumption: there is exactly one ground-truth target commit that satisfies the developer’s query.

In real-world software maintenance scenarios, engineering issues are frequently distributed across multiple disjoint commits, such as a primary bug fix followed by successive regression corrections or localized configuration updates. Because our current metric configuration only tracks the relative position of the single, mutated source commit, it penalizes a re-ranker for bubbling up a highly relevant, conceptually related historical patch if that patch does not match the exact target hash. Future iterations of this benchmark framework would benefit from incorporating multi-label evaluation metrics, such as Normalized Discounted Cumulative Gain ($NDCG$).

Our infrastructure transition from local execution to cloud-hosted API clusters introduces operational variables with regard to latency and replication stability. The local CPU execution environment encountered a major processing bottleneck, forcing reliance on commercial endpoints (`gemini-2.5-flash`) to complete the 25-query listwise benchmark loop without triggering client-side timeouts.

Relying on closed, commercial API layers introduces non-deterministic risks. Cloud providers frequently apply unannounced backend updates, parameter quantizations, or routing variations to their infrastructure to optimize server loads. These backend changes can introduce minor variations in the token generation patterns of listwise models, even when the temperature parameter is locked at 0.0.

VII. CONCLUSION AND FUTURE WORK

We investigated a multi-stage information retrieval architecture designed to optimize version control history mining across heterogeneous open-source software repositories. By implementing a cascading pipeline that couples a high-recall lexical index (BM25) with high-precision semantic re-ranking layers, we addressed the traditional vocabulary mismatch problem while maintaining low system latency.

Our empirical evaluations across the `google/gson`, `spring-boot`, and `dbeaver` ecosystems demonstrate clear architectural trade-offs between discriminative and generative neural models. While listwise generative language models

offer robust global reasoning capabilities, they suffer from significant performance degradation, contextual dilution, and positional biases when processing dense or boilerplate-heavy metadata structures. In contrast, dense Cross-Encoder frameworks that utilize joint token-level self-attention consistently achieved superior precision, delivering a $MRR@10$ score of up to 0.8800 and significantly outperforming both the lexical baseline and the generative sorting strategies.

This project aims to make commit histories more accessible and actionable for developers navigating large software repositories. For onboarding engineers, this tool provides vital context to grasp historical architectural decisions and confidently implement new features. For senior engineers, it streamlines code reviews and feature development by surfacing the deeper context embedded within the repository’s evolution. Ultimately, this work treats commit messages not merely as logs, but as essential software artifacts, demonstrating that maintaining a meaningful commit history is fundamental to long-term documentation and codebase health.

A. Future Work

Future work will focus on expanding this multi-stage retrieval architecture along three distinct trajectories, moving away from isolated, single-commit queries toward holistic codebase synthesis.

First, the architecture will be expanded to support multi-commit and group-level retrieval. Rather than evaluating commits in isolation, future iterations will focus on aggregating historical sequences and multi-commit bug fixes into unified contextual blocks. Treating related commits as cohesive transactional units will allow the pipeline to accurately index and retrieve complete regression sequences and distributed architectural changes that span multiple changesets.

Currently, the retrieval pipeline is restricted to searching exclusively within the textual summary and body of commit messages. A significant architectural improvement would involve indexing the actual source code diffs associated with each commit. Integrating the underlying code changes would provide richer semantic context, bridging the gap when text descriptions are vague or incomplete.

The scope of the ingestion engine will extend from linear histories to complex multi-branch environments. This trajectory will investigate specialized indexing strategies designed to traverse divergent git branches, pull requests, and parallel development forks simultaneously. Enabling cross-branch search will allow developers to evaluate conceptual proximity and track code evolution across active, concurrent versions of the codebase rather than restricting the system’s visibility to a single primary branch.

Finally, the output of this enhanced retrieval pipeline will be integrated into a fully realized Retrieval-Augmented Generation (RAG) framework to power downstream Large Language Models. By feeding the retrieved code blocks, diff logic, and historical commit contexts directly into the context windows of specialized LLMs, the system can significantly mitigate model hallucination.